

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"Белгородский государственный технологический университет им. В. Г.
Шухова"
(БГТУ им. В.Г. Шухова)



Институт энергетики, информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники
и автоматизированных систем

Лабораторная работа № 6
по дисциплине: Исследование операций
по теме: Нахождение седловой точки в смешанных
стратегиях для матричной игры с нулевой суммой

Выполнил: студент группы ПВ-221
Дорошенко Владимир Юрьевич
Проверил: ст. преподаватель:
Вирченко Ю.В.

Белгород 2024

Цель работы: Освоить метод нахождения седловой точки в смешанных стратегиях с помощью построения пары двойственных задач ЛП.

Задания для подготовки к работе

1. Изучить основные понятия теории матричных игр двух игроков с нулевой суммой, анализ игры в чистых стратегиях, понятие смешанной стратегии и седловой точки в смешанных стратегиях, а также метод нахождения седловой точки в смешанных стратегиях с помощью построения пары двойственных задач ЛП.
2. Составить и отладить программу для нахождения седловой точки игры с помощью решения пары симметрично двойственных задач ЛП.
3. Для подготовки тестовых данных решить вручную одну из следующих ниже задач.

7.

$$\begin{pmatrix} 7 & 10 & 27 & 9 & 12 \\ 6 & 21 & 5 & 8 & 1 \end{pmatrix}$$

Первый этап – анализ на чистые стратегии

7	10	27	9	12	7
6	21	5	8	1	1
7	21	27	9	8	

max из min = 7

min из max = 7

Игра имеет решение в чистых стратегиях

Второй этап – составление пары двойственных задач

	q1	q2	q3	q4	q5
p1	7	10	27	9	12
p2	6	21	5	8	1

→

$p(p_1; p_2)$ – стратегия первого игрока

→

$q(q_1; q_2; q_3; q_4; q_5)$ – стратегия второго игрока

V -цена сырья

$$z = V \rightarrow \max$$

$$f = u \rightarrow \min$$

$$\begin{cases} 7p_1 + 6p_2 \geq V \\ 10p_1 + 21p_2 \geq V \\ 27p_1 + 5p_2 \geq V \\ 9p_1 + 8p_2 \geq V \\ 12p_1 + p_2 \geq V \\ p_1 + p_2 = 1 \\ p_i \geq 0 \end{cases}$$

$$\begin{cases} 4q_1 + 10q_2 + 27q_3 + 9q_4 + 12q_5 \leq u \\ 6q_1 + 21q_2 + 5q_3 + 8q_4 + q_5 \leq u \\ q_1 + q_2 + q_3 + q_4 + q_5 = 1 \\ q_j \geq 0 \end{cases}$$

Введем обозначение: $x_i = \frac{p_i}{V}$; $y_i = \frac{q_i}{u}$

$$p_i = x_i V$$

$$1 = p_1 + p_2 = (x_1 + x_2) V$$

$$\frac{1}{V} = x_1 + x_2$$

$$Z = X_1 + X_2 \rightarrow \min$$

$$\begin{cases} 7X_1 + 6X_2 \geq 1 \\ 10X_1 + 21X_2 \geq 1 \\ 27X_1 + 5X_2 \geq 1 \\ 9X_1 + 8X_2 \geq 1 \\ 12X_1 + X_2 \geq 1 \\ X_i \geq 0 \end{cases}$$

$$P = y_1 + y_2 + y_3 + y_4 + y_5 \rightarrow \max$$

$$\begin{cases} 7y_1 + 6y_2 + 27y_3 + 9y_4 + 12y_5 \leq 1 \\ 10y_1 + 21y_2 + 5y_3 + 8y_4 + y_5 \leq 1 \\ y_i \geq 0 \end{cases}$$

Третий этап - решаем задачу из двойственных данных

$$P = y_1 + y_2 + y_3 + y_4 + y_5 \rightarrow \max$$

Составим симплекс-таблицу

B	L	y_1	y_2	y_3	y_4	y_5	y_6	y_7
y_6	1	7	10	27	9	12	1	0
y_7	1	6	21	5	8	1	0	1
f	0	-1	-1	-1	-1	-1	0	0

Итерация №0.

Базис	В	x_1	x_2	x_3	x_4	x_5	x_6	x_7	min
x_6	1	7	10	27	3	12	1	0	1/27
x_7	1	6	21	5	8	1	0	1	1/5
F(X1)	-2M	-1-13M	-1-31M	-1-32M	-1-11M	-1-13M	0	0	

Получаем новую симплекс-таблицу:

Базис	В	x_1	x_2	x_3	x_4	x_5	x_6	x_7
x_3	1/27	7/27	10/27	1	1/9	4/9	1/27	0
x_7	22/27	127/27	517/27	0	67/9	-11/9	-5/27	1
F(X1)	$\frac{1}{27}-\frac{22}{27}M$	$-\frac{20}{27}-\frac{127}{27}M$	$-\frac{17}{27}-\frac{517}{27}M$	0	$-\frac{8}{9}-\frac{67}{9}M$	$-\frac{5}{9}+\frac{11}{9}M$	$\frac{1}{27}+\frac{32}{27}M$	0

Итерация №1.

Базис	В	x_1	x_2	x_3	x_4	x_5	x_6	x_7	min
x_3	1/27	7/27	10/27	1	1/9	4/9	1/27	0	1/10
x_7	22/27	127/27	517/27	0	67/9	-11/9	-5/27	1	2/47
F(X2)	$\frac{1}{27}-\frac{22}{27}M$	$-\frac{20}{27}-\frac{127}{27}M$	$-\frac{17}{27}-\frac{517}{27}M$	0	$-\frac{8}{9}-\frac{67}{9}M$	$-\frac{5}{9}+\frac{11}{9}M$	$\frac{1}{27}+\frac{32}{27}M$	0	

Получаем новую симплекс-таблицу:

Базис	В	x_1	x_2	x_3	x_4	x_5	x_6	x_7
x_3	1/47	87/517	0	1	-17/517	22/47	21/517	-10/517
x_2	2/47	127/517	1	0	201/517	-3/47	-5/517	27/517
F(X2)	$\frac{3}{47}$	$-\frac{303}{517}$	0	0	$-\frac{333}{517}$	$-\frac{28}{47}$	$\frac{16}{517}+M$	$\frac{17}{517}+M$

Итерация №2.

Базис	В	x_1	x_2	x_3	x_4	x_5	x_6	x_7	min
x_3	1/47	87/517	0	1	-17/517	22/47	21/517	-10/517	-
x_2	2/47	127/517	1	0	201/517	-3/47	-5/517	27/517	22/201
F(X3)	$\frac{3}{47}$	$-\frac{303}{517}$	0	0	$-\frac{333}{517}$	$-\frac{28}{47}$	$\frac{16}{517}+M$	$\frac{17}{517}+M$	

Получаем новую симплекс-таблицу:

Базис	В	x_1	x_2	x_3	x_4	x_5	x_6	x_7
x_3	5/201	38/201	17/201	1	0	31/67	8/201	-1/67
x_4	22/201	127/201	517/201	0	1	-11/67	-5/201	9/67
F(X3)	$\frac{9}{67}$	$-\frac{12}{67}$	$\frac{111}{67}$	0	0	$-\frac{47}{67}$	$\frac{1}{67}+M$	$\frac{8}{67}+M$

Итерация №3.

Базис	В	x_1	x_2	x_3	x_4	x_5	x_6	x_7	min
x_3	5/201	38/201	17/201	1	0	31/67	8/201	-1/67	5/93
x_4	22/201	127/201	517/201	0	1	-11/67	-5/201	9/67	-
F(X4)	$\frac{9}{67}$	$-\frac{12}{67}$	$\frac{111}{67}$	0	0	$-\frac{47}{67}$	$\frac{1}{67}+M$	$\frac{8}{67}+M$	

Получаем новую симплекс-таблицу:

Базис	В	x_1	x_2	x_3	x_4	x_5	x_6	x_7
x_5	5/93	38/93	17/93	67/31	0	1	8/93	-1/31
x_4	11/93	65/93	242/93	11/31	1	0	-1/93	4/31
F(X4)	$\frac{16}{93}$	$\frac{10}{93}$	$\frac{166}{93}$	$\frac{47}{31}$	0	0	$\frac{7}{93}+M$	$\frac{3}{31}+M$

1. Проверка критерия оптимальности.

Среди значений индексной строки нет отрицательных. Поэтому эта таблица определяет оптимальный план задачи.

Окончательный вариант симплекс-таблицы:

Базис	В	x_1	x_2	x_3	x_4	x_5	x_6	x_7
x_5	5/93	38/93	17/93	67/31	0	1	8/93	-1/31
x_4	11/93	65/93	242/93	11/31	1	0	-1/93	4/31
F(x_5)	$16/93$	$10/93$	$166/93$	$47/31$	0	0	$7/93+M$	$3/31+M$

Так как в оптимальном решении отсутствуют искусственные переменные (они равны нулю), то данное решение является допустимым.

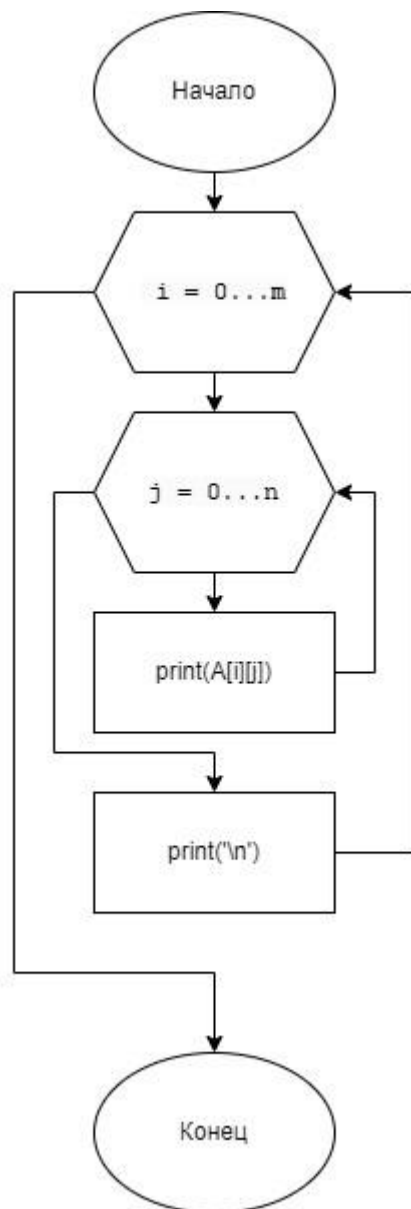
Оптимальный план можно записать так:

$$x_1 = 0, x_2 = 0, x_3 = 0, x_4 = 11/93, x_5 = 5/93$$

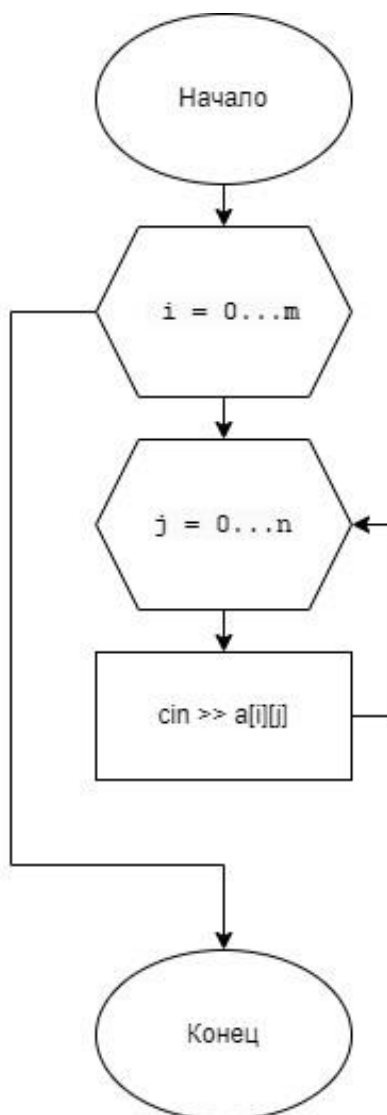
$$F(X) = 1 \cdot 0 + 1 \cdot 0 + 1 \cdot 0 + 1 \cdot 11/93 + 1 \cdot 5/93 = 16/93$$

Блок – схемы:

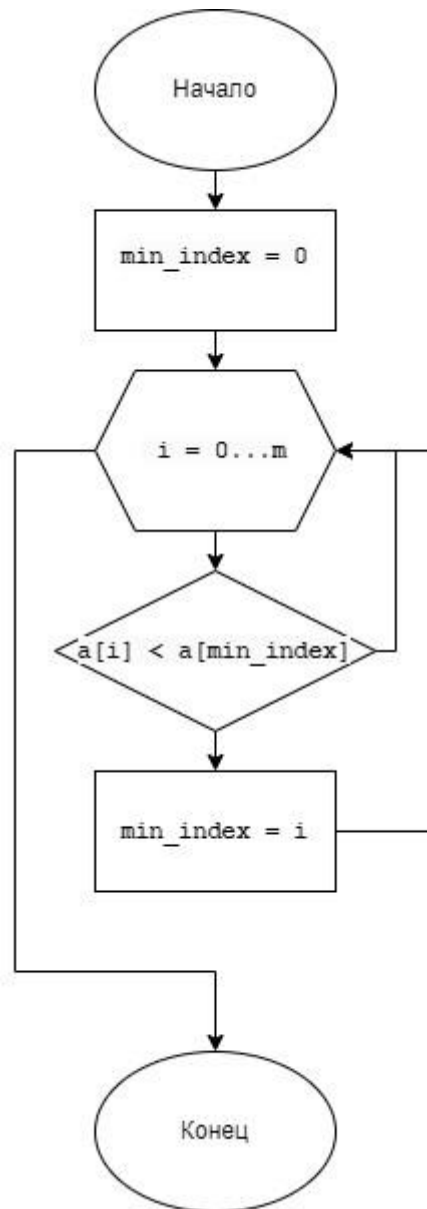
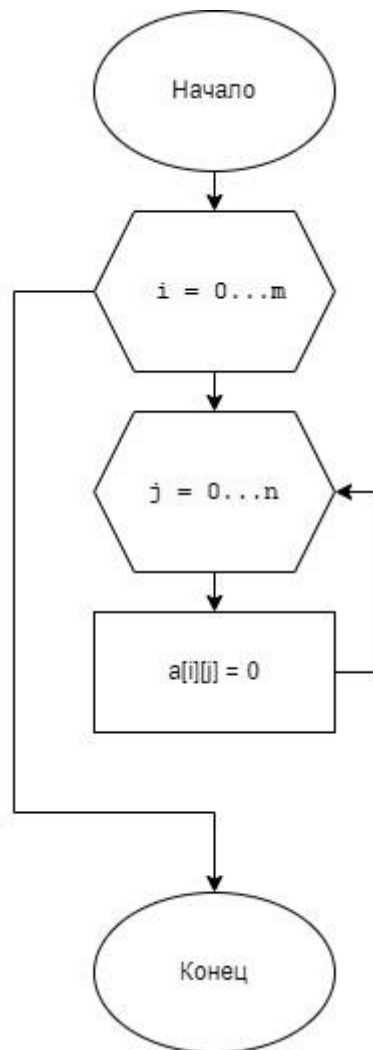
//Вывод матрицы на экран



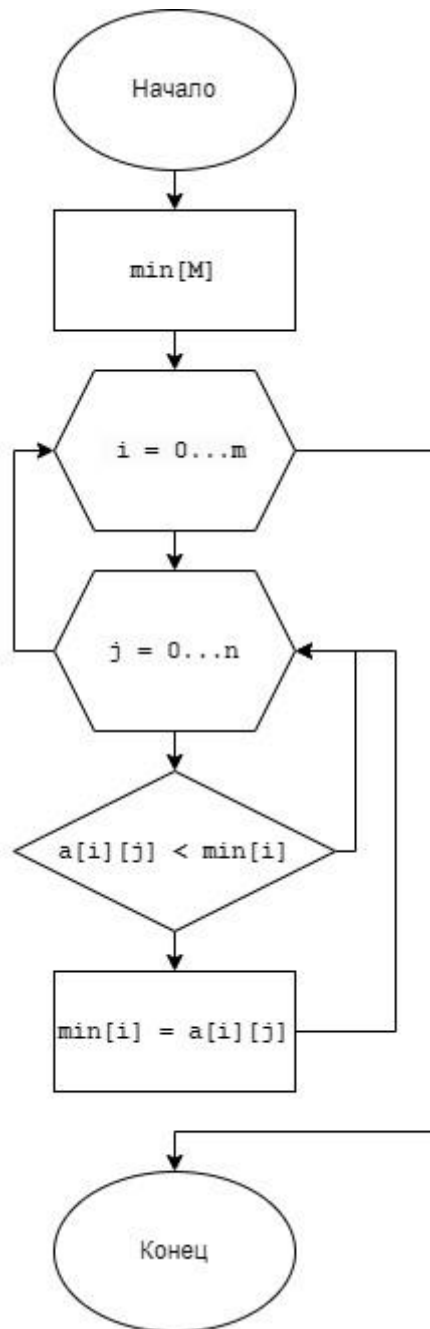
//Функция создаёт матрицу и вписывает туда значения



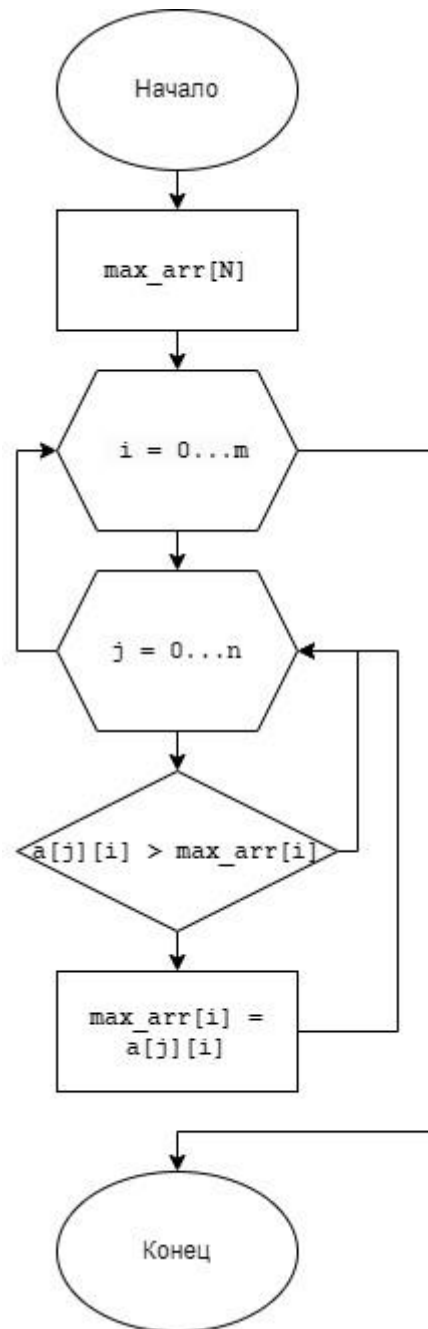
//Создание нулевой матрицы //Поиск минимального индекса



//Поиск минимального максимума



//Поиск максимального минимума



Код программы:

```
#include <stdio.h>

#include <limits.h>

#include <iostream>

#include <locale.h>

#include<stdlib.h>

using namespace std;

void ShowMat(double** A, int m, int n) {
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++)
            cout << A[i][j] << "\t";
        cout << "\n";
    }
}

void Operate(double** A, int m, int n, int a, int b) {
    double p;
    double** B;
    B = (double**)malloc((m + 1) * sizeof(double*));
    for (int i = 0; i <= m; i++)
        B[i] = (double*)malloc((n + 1) * sizeof(double));
    for (int i = 0; i <= m; i++)
        for (int j = 0; j <= n; j++) B[i][j] = A[i][j];

    p = B[a][b];
    A[a][b] = 1 / p;
    for (int i = 0; i <= m; i++)
        if (i != a)
            A[i][b] = -1 * B[i][b] / p;
    for (int j = 0; j <= n; j++)
        if (j != b)
            A[a][j] = B[a][j] / p;
    for (int i = 0; i <= m; i++) {
        for (int j = 0; j <= n; j++) {
            if ((i != a) && (j != b))
                A[i][j] = ((B[a][b] * B[i][j]) - (B[a][j] * B[i][b])) / p;
        }
    }
}
```

```

    }
}

double Primal(double** A, int* M, int m, int n, int im) {
    int J = -1, I;
    double min = 0, PI;
    for (int j = 0; j < n; j++) if (A[m][j] < min) { min = A[m][j]; J = j; }
    double sum = -1, ratio;
    for (int i = 0; i < m; i++) {
        if (A[i][J] > 0) {
            ratio = A[i][n] / A[i][J];
            if ((sum == -1) || (sum > ratio)) {
                sum = ratio;
                I = i;
            }
        }
    }
    if (im == 1) {
        PI = A[I][n] * A[m][J] / A[I][J];
        if (PI < 0)
            PI *= -1;
        //cout << "PI=" << PI << "\n";
        return PI;
    }
    Operate(A, m, n, I, J);
    int temp;
    temp = M[I + n];
    M[I + n] = M[J];
    M[J] = temp;
    return 0;
}

```

```

double Dual(double** A, int* M, int m, int n, int im) {
    int I = -1, J;
    double min = 0, DI;
    for (int i = 0; i < m; i++) if (A[i][n] < min) { min = A[i][n]; I = i; }
    double sum = -1, ratio;
    for (int j = 0; j < m; j++) {

```

```

        if (A[I][j] < 0) {
            ratio = A[m][j] / A[I][j];
            if ((sum == -1) || (sum > ratio)) { sum = ratio; J = j; }
        }
    }

    if (im == 1) {
        DI = A[I][n] * A[m][J] / A[I][J];
        if (DI < 0)
            DI *= -1;
        //cout << "DI=" << DI << "\n";
        return DI;
    }

    Operate(A, m, n, I, J);

    int temp;
    temp = M[I + n];
    M[I + n] = M[J];
    M[J] = temp;
    return 0;
}

void simplex(double** A, int* M, int m, int n) {
    int c1 = 0, c2 = 0;
    for (int i = 0; i < m; i++)
        if (A[i][n] < 0)
            c1 = 1;
    for (int j = 0; j < n; j++)
        if (A[m][j] < 0)
            c2 = 1;
    if ((c1 == 0) && (c2 == 0))
        return;
    else if (c1 == 0) {
        //cout << "Primal simplex" << endl;
        Primal(A, M, m, n, 0);
    }
    else if (c2 == 0) {
        //cout << "Dual simplex" << endl;
        Dual(A, M, m, n, 0);
    }
}

```

```

else {
    double PI, DI;

    PI = Primal(A, M, m, n, 1);
    DI = Dual(A, M, m, n, 1);

    if (PI >= DI) {
        //cout << "Primal simplex" << endl;

        Primal(A, M, m, n, 0);
    }

    else {
        //cout << "Dual simplex" << endl;

        Dual(A, M, m, n, 0);
    }
}

simplex(A, M, m, n);
}

```

```

void Result(double** A, int* M, int m, int n) {
    double c = 1 / A[m][n];

    cout << "Цена игры:\nV = " << c << endl;

    double* b = new double[m + n];

    for (int i = 0; i < m; i++) {
        b[M[i] + n] = A[i][n];
    }

    for (int i = 0; i < n; i++) {
        b[M[i]] = 0;
    }

    double* a = new double[m + n];

    a[0] = A[m][1];
    a[1] = A[m][3];
    a[2] = A[m][0];
    a[3] = 0;

    cout << "Оптимальная стратегия игрока 2\n";

    cout << "(";

    for (int i = 1; i < n; i++)
        cout << b[i] * c << ", ";

    cout << b[n] * c << ")\n";

    cout << "Оптимальная стратегия игрока 1\n";
}

```

```

        cout << "(";

        for (int i = 0; i < n - 1; i++)
            cout << a[i] * c << ", ";

        cout << a[n - 1] * c << ")\n";
    }

#define N 100
#define M 100

void input_matr(float a[M][N], int m, int n)
{
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            cin >> a[i][j];
}

void null_matr(float a[M][N], int m, int n)
{
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            a[i][j] = 0;
}

void output_matr(float a[M][N], int m, int n)
{
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++)
            printf("%lg ", a[i][j]);

        printf("\n");
    }
}

void output_arr(float a[M], int m)
{
    for (int i = 0; i < m; i++)
        printf("%lg ", a[i]);

    printf("\n");
}

```

```

}

void out_arr(int a[M], int m)
{
    for (int i = 0; i < m; i++)
        printf("%d ", a[i]);
    printf("\n");
}

//проверка на то, можно ли использовать симплекс метод в чистом виде
int is_exam_matr(float a[M][N], int m, int n, int base[M])
{
    int fl;
    int t = 0;
    for (int i = 0; i < m - 1; i++) { //проверка на то имеет ли система базисный вид
        for (int j = 0; j < n; j++) {
            fl = 1;
            if (a[i][j]) {
                for (int z = 0; z < m; z++) {
                    if (a[z][j] != 0 && z != i && fl == 1)
                        fl = 0;
                }
                if (fl == 1) {
                    base[i] = j;
                    t = 1;
                }
            }
        }
        if (t == 0)
            return 0;
        t = 0;
    }

    for (int i = 0; i < m; i++) { //проверка что при делении базисной на свободный
        коэфф получается положительное число
        if (a[i][n] / a[i][base[i]] < 0)
            return 0;
    }
    return 1;
}

```



```
}
```

```
int search_min_index(float a[M], int m)
```

```
{
```

```
    int min_index = 0;
```

```
    for (int i = 0; i < m; i++) {
```

```
        if (a[i] < a[min_index])
```

```
            min_index = i;
```

```
    }
```

```
    return min_index;
```

```
}
```

```
void simplex_method(float a[M][N], int m, int n, int base[M])
```

```
{
```

```
    int fl = 1;
```

```
    int index;
```

```
    float tarr[M];
```

```
    while (fl) {
```

```
        index = search_min_index(a[m - 1], n);
```

```
        if (a[m - 1][index] >= 0)
```

```
            break;
```

```
        for (int i = 0; i < m - 1; i++) {
```

```
            float t = a[i][n - 1] / a[i][index];
```

```
            if (t > 0)
```

```
                tarr[i] = t;
```

```
            else
```

```
                tarr[i] = 10000;
```

```
        }
```

```
        //индекс базисной переменной в массиве базисных которую нужно заменить на  
        переменную с индексом index
```

```
        int t_new = search_min_index(tarr, m - 1);
```

```
        base[t_new] = index;
```

```
        float del = a[t_new][index];
```

```
        for (int i = 0; i < n; i++)
```

```
            a[t_new][i] /= del;
```

```
        for (int i = 0; i < m; i++) {
```

```

        del = a[i][index];

        for (int j = 0; j < n; j++) {
            if (i != t_new)
                a[i][j] -= del * a[t_new][j];
        }
    }
}

cout << "z(min) = f(max) = %lg\n" << a[m - 1][n - 1];
for (int i = 0; i < m - 1; i++)
    cout << "y%d = %lg\n" << base[i] + 1, a[i][n - 1];
}

//заполнение массива максимальными возможными
void in_max_arr(float a[M], int n)
{
    for (int i = 0; i < n; i++)
        a[i] = INT_MAX;
}

float search_max(float arr[M], int n)
{
    float max = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > max)
            max = arr[i];
    }
    return max;
}

//нахождение нижней цены игры
float func_minmax(float a[M][N], int m, int n)
{
    float min[M];
    in_max_arr(min, m);
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            if (a[i][j] < min[i])

```

```

        min[i] = a[i][j];
    }
}

return(search_max(min, m));
}

//заполнение массива минимальными значениями
void in_min_arr(float arr[N], int n)
{
    for (int i = 0; i < n; i++)
        arr[i] = INT_MIN;
}

float search_min(float arr[N], int n)
{
    float min = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] < min)
            min = arr[i];
    }
    return min;
}

//нахождение верхней цены игры
float func_maxmin(float a[M][N], int m, int n)
{
    float max_arr[N];
    in_min_arr(max_arr, n);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (a[j][i] > max_arr[i]) {
                max_arr[i] = a[j][i];
            }
        }
    }
}

```

```

        return(search_min(max_arr, n));
    }

    //стратегии игроков
    void gener_strategy(float a[M][N], int m, int n, int base[M])
    {
        for (int i = 0; i < m - 1; i++) {
            if (base[i] < m - 2) {
                printf("x%d = y%d = %lg\n", base[i] + 1, base[i] + 1, a[i][n - 1]);
                printf("p%d = %lg\n", base[i] + 1, a[i][n - 1] / a[m - 1][n - 1]);
            }
        }

        printf("Цена игры по симплекс таблице = %lg\n", 1. / a[m - 1][n - 1]);
    }

    int main() {
        setlocale(LC_ALL, "rus");

        int n;

        int m = 4;

        printf("введите количество столбцов в платежной матрице: \n");

        cin >> n;

        printf("введите платежную матрицу\n");

        float a[M][N] = { 0 };

        null_matr(a, m, n);

        input_matr(a, m, n);

        float minmax = func_minmax(a, m, n);

        float maxmin = func_maxmin(a, m, n);

        printf("Нижняя цена = %lg\nВерхняя цена = %lg\n", minmax, maxmin);

        if (minmax == maxmin) {
            n += m; //добавляем две переменные(по одной в каждую строку)

            for (int i = 0; i < m; i++)
                a[i][n - m + i] = 1;

            n++;

            for (int i = 0; i < m; i++)//заполняем свободные
                a[i][n - 1] = 1;
        }
    }
}

```

```

m++;

for (int i = 0; i < n - m; i++) //заполняем целевую функцию
    a[m - 1][i] = -1;

for (int i = n - m; i < n; i++)
    a[m - 1][i] = 0;

printf("Первая симплекс-таблица по двойственной задаче:\n");

output_matr(a, m, n);

int base[M]; //набор базисных

printf("\n");

if (is_exam_matr(a, m, n, base)) { //если система подходит по условиям,
сформирован набор базисных
    simplex_method(a, m, n, base);
    output_matr(a, m, n);
    printf("Цена игры(по верхней и нижней) = %lg\n", minmax);
    gener_strategy(a, m, n, base);
}

else
    printf("Система не удовлетворяет условиям, для решения Симплекс
метода\n");
}

else {
    printf("Игра в чистых стратегиях невозможна\n");

    double* Z, ** A;

    int* B;

    Z = (double*)malloc(n * sizeof(double));

    A = (double**)malloc((m + 1) * sizeof(double*));
    for (int i = 0; i <= m; i++)
        A[i] = (double*)malloc((n + 1) * sizeof(double));
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            A[i][j] = a[i][j];
    for (int i = 0; i < m; i++)
        A[i][n] = 1;
    B = (int*)malloc((n + m) * sizeof(int));
    for (int i = 0; i < m + n; i++)
        B[i] = i + 1;
}

```

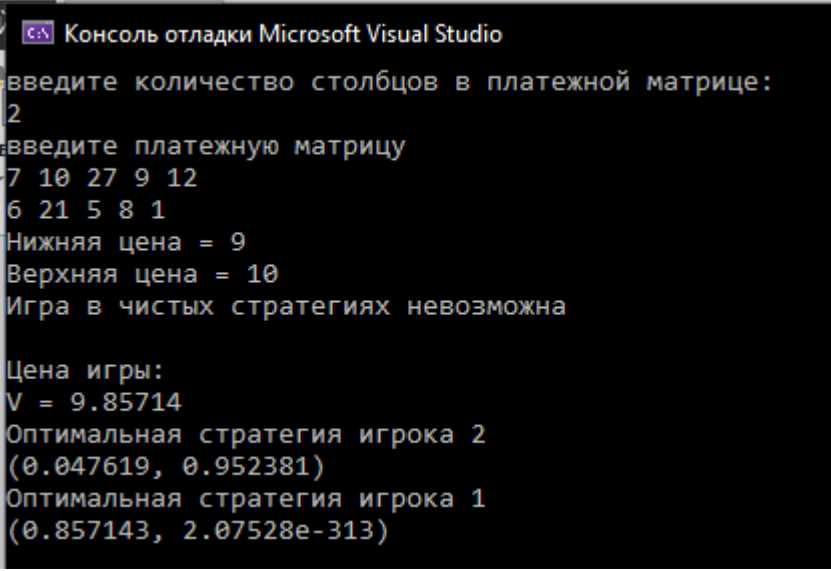
```

        for (int i = 0; i < n; i++)
            Z[i] = 1;
        for (int i = 0; i < n; i++)
            A[m][i] = -1 * Z[i];
        A[m][n] = 0;
        simplex(A, B, m, n);

        cout << "\n";
        Result(A, B, m, n);
        cout << "\n\n";
    }

    return 0;
}

```



Консоль отладки Microsoft Visual Studio

```

Введите количество столбцов в платежной матрице:
2
Введите платежную матрицу
7 10 27 9 12
6 21 5 8 1
Нижняя цена = 9
Верхняя цена = 10
Игра в чистых стратегиях невозможна

Цена игры:
V = 9.85714
Оптимальная стратегия игрока 2
(0.047619, 0.952381)
Оптимальная стратегия игрока 1
(0.857143, 2.07528e-313)

```

Вывод

Метод нахождения седловой точки в смешанных стратегиях с использованием двойственных задач линейного программирования представляет собой мощный инструмент анализа в теории игр. Он позволяет не только определить седловую точку, но и вычислить оптимальные стратегии для обоих игроков и их выигрыши.

Суть метода заключается в построении двух двойственных задач, которые соответствуют исходной игре. Первая задача связана с максимизацией выигрыша игрока А, а вторая - с минимизацией выигрыша игрока В. Решение этих задач приводит к нахождению так называемой "цены игры" - значение, которое игрок А гарантированно выиграет, если он выбирает оптимальную стратегию, а также к оптимальным стратегиям обоих игроков.

Этот метод позволяет не только определить седловую точку и выигрыши игроков, но и понять, какие стратегии им следует выбирать для достижения оптимальных результатов в игре. Благодаря своей математической основе и точности, метод двойственных задач ЛП находит применение в различных областях, включая экономику, управление, стратегическое планирование и теорию принятия решений.